CS571: Programming Languages
Final Exam Practice Questions Solution

Problem 1 Given the following regular expression:

```
(a(b|c))*d*
```

1. Write down the set of strings recognized by the following regular expressions. If the set is infinite, write down all strings that have 3 or fewer characters.

   Solution: {ϵ, `d`, `ab`, `ac`, `dd`, `abd`, `acd`, `ddd`, `...`}

2. How many strings of length 4 will be recognized by this regular expression?

   Solution: 7

3. Write down a regular expression that captures all **non-empty binary** strings of **even** lengths that start and end with the **same** bit.

   Solution: `0([01][01])*0 | 1([01][01])*1`

Problem 2 Consider the following context free grammar:

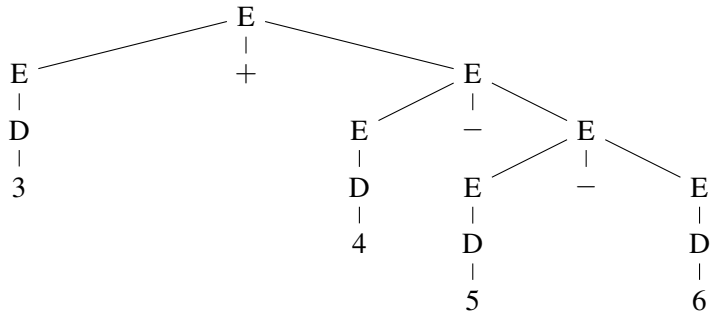
$$E \to E + E \mid E - E \mid D \mid (E)$$
$$D \to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

where $+, -, (,)$ and digits $0, 1, \ldots, 9$ are terminals.

a) Write down the **leftmost derivation** and draw a corresponding *concrete* parse tree for the string "$3+4-5-6$". **Show the detailed derivation in your answer**.

Solution: Below is one solution. There are other correct solutions.

$E \Rightarrow E + E \Rightarrow D + E \Rightarrow 3 + E \Rightarrow 3 + E - E \Rightarrow 3 + D - E \Rightarrow 3 + 4 - E \Rightarrow 3 + 4 - E - E \Rightarrow 3 + 4 - D - E \Rightarrow 3 + 4 - 5 - E \Rightarrow 3 + 4 - 5 - D \Rightarrow 3 + 4 - 5 - 6$



b) Show why this grammar is ambiguous.

Solution: Show an example of a string having two parse trees, e.g., "1+2+3".

c) Rewrite the grammar to an equivalent one which is *unambiguous* and in which $+$ is *left associative*, $-$ is *right associative*, and $+$ has *lower precedence* than $-$.

Solution:

$$E \to E + F \mid F$$
$$F \to G - F \mid G$$
$$G \to D \mid (E)$$
$$D \to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Problem 3

a) Fully evaluate the following λ-term so that no further β-reduction is possible. You must show detailed steps to earn credit for this problem.

$$(\lambda x\ y.\ x\ y)\ (\lambda x.\ y)\ u$$

Solution:

$$\begin{aligned}
&((\lambda x\ y.\ x\ y)\ (\lambda x.\ y))\ u \\
&= ((\lambda x.\ (\lambda y.\ x\ y))\ (\lambda x.\ y))\ u && \text{(desugar term } \lambda x\ y.\ x\ y) \\
&= ((\lambda x.\ (\lambda z.\ x\ z))\ (\lambda x.\ y))\ u && (\alpha - \text{reduction}) \\
&= (\lambda z.\ (\lambda x.\ y)\ z)\ u && (\beta - \text{reduction}) \\
&= (\lambda x.\ y)\ u && (\beta - \text{reduction}) \\
&= y && (\beta - \text{reduction})
\end{aligned}$$

b) Recall the following definitions of Church Encoding we defined in our class,

$$\underline{n} \triangleq \lambda f\ z.\ f^n\ z \qquad \underline{0} \triangleq \lambda f\ z.\ z \qquad \underline{1} \triangleq \lambda f\ z.\ f\ z$$

$$\texttt{SUCC} \triangleq \lambda n.\ \lambda f\ z.\ f\ (n\ f\ z) \qquad \texttt{PLUS} \triangleq \lambda m\ n.\ m\ \texttt{SUCC}\ n$$

$$\texttt{PAIR} \triangleq \lambda x\ y\ f.f\ x\ y \qquad \texttt{LEFT} \triangleq \lambda p.p\ \lambda x\ y.x \qquad \texttt{RIGHT} \triangleq \lambda p.p\ \lambda x\ y.y$$

c) Fully evaluate $\texttt{RIGHT}\ (\texttt{PAIR}\ \underline{1}\ \underline{0})$ so that no further β-reduction is possible. Show detailed steps of your computation.

Solution:

$$\begin{aligned}
&\texttt{RIGHT}\ (\texttt{PAIR}\ \underline{1}\ \underline{0}) \\
&= (\lambda p.p\ \lambda x\ y.y)\ (\texttt{PAIR}\ \underline{1}\ \underline{0}) && (\text{definition of } \texttt{RIGHT}) \\
&= (\texttt{PAIR}\ \underline{1}\ \underline{0})\ \ (\lambda x\ y.y) && (\beta - \text{reduction}) \\
&= ((\lambda x\ y\ f.f\ x\ y)\ \ \underline{1}\ \ \underline{0})\ \ (\lambda x\ y.y) && (\text{definition of } \texttt{PAIR}) \\
&= (\lambda f.f\ \underline{1}\ \underline{0})\ \ (\lambda x\ y.y) && (\beta - \text{reduction}) \\
&= (\lambda x\ y.y)\ \underline{1}\ \underline{0} && (\beta - \text{reduction}) \\
&= \underline{0} && (\beta - \text{reduction})
\end{aligned}$$

Problem 4 Consider the following pseudo-code with higher-order function support. Assume that the language has one scope for each function, and it allows nested functions (hence, nested scopes).

```
function A(x, P) {
  function B() {
    print(x);
  }
  function C() {
    int x = 4;
    print(x);
  }
  if (x>1){
    P();
  } else{
    A(2,B);
  }
}
A(1,C);
```

a) Draw two symbol tables: one for the global scope and one for function `A` in the given program. Assume each table contains two columns: name and kind (e.g., var, fun, para).

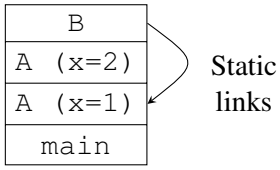
Solution: The symbol tables are as follows:

Global	
Name	Kind
A	fun

A	
Name	Kind
x	para
P	para
B	fun
C	fun

b) What does this program print if the language uses *static scoping* with *deep binding*? You must draw the run time stack with static links to show how you come to the conclusion.

Solution: The program will print 1.



Problem 5 Consider the following program. Write down what will be printed out when the parameters are passed (1) by value, (2) by reference, and (3) by value return.

```
int x = 5, y = 6;
void foo(int a, int b) {
  x = a+b;
  b = a+a;
  return a+b;
}
main () {
  print foo(x, y);
  print(x);
  print(y);
}
```

Solution:

(1) 15, 11, 6 (call by value)

(2) 33, 11, 22 (call by reference)

(3) 15, 5, 10 (call by value return)

Problem 6

a) What is the value of the following racket expression?

```
#lang racket
(define x 1)
(define y 2)
(let* [(x 3)
       (y (+ x y))]
  (let [(x 4)
        (y (+ x y))]
    (* x y)
    ))
```

Solution: 32

b) Given the following racket code,what is the output of the program?

```
#lang racket
(define (mystery t) (foldl cons null t))
(mystery '(1 2 3))
```

Solution:

```
'(3 2 1)
```

c) We define the size of a value as follows: the size of a non-list value is 1; the size of a list value is the sum of the sizes of its elements; the size of an empty list is 0. Implement a recursive function `sizeOf` that takes a value and returns the size of that value. For example:

```
(sizeOf 2) ; returns 1
(sizeOf '(1 ("abc" ()) 2)) ; returns 3
(sizeOf '()) ; returns 0
(sizeOf '(a b (c d e (f g)) ((h i) (j)))) ; returns 10
```

Use `(list? v)` to check if a value `v` is a list.

Solution:

```
(define (sizeOf t)
  (if (list? t)
      (if (null? t)
          0
          (+ (sizeOf (car t)) (sizeOf (cdr t))))
      1))
```

Problem 7 Consider the following C declaration:

```
struct S1 {int a; double b; char c;};
union  S2 {float a; char b;};
union U {
    struct S1 s1;
    union S2 s2;
} u;
union U arr[5][10];
```

Assume the machine has 1-byte characters, 4-byte integers, 4-byte floating numbers, and 8-byte double-precision floating numbers. Assume the compiler does not reorder the fields and the compiler might add padding in between fields and at the end of the struct to ensure that all fields start at a memory location with proper alignment. The size of the struct will also be a multiple of its largest alignment requirement.

a) What is the size of a `struct S1` value? **Solution**: 24 bytes.

a	a	a	a
padding	padding	padding	padding
b	b	b	b
b	b	b	b
c	padding	padding	padding
padding	padding	padding	padding

b) What is the size of a `union S2` value?

Solution: 4 bytes.

a/b	a	a	a

c) What is the size of a `union U` value `u`? **Solution**: $\max(24, 4) = 24$ bytes.

s1/s2	s1/s2	s1/s2	s1/s2
s1	s1	s1	s1
s1	s1	s1	s1
s1	s1	s1	s1
s1	s1	s1	s1
s1	s1	s1	s1

d) If the memory address of `u` starts from 3000, what is the start address of `u.s1.b`? What is the start address of `u.s2.b`?

Solution: The start address of u.s1.b is $\boxed{3008}$, the start address of u.s2.b is $\boxed{3000}$.

3000	u.s1.a/u.s2.a/u.s2.b	u.s1.a/u.s2.a	u.s1.a/u.s2.a	u.s1.a/u.s2.a
	padding	padding	padding	padding
3008	u.s1.b	u.s1.b	u.s1.b	u.s1.b
	u.s1.b	u.s1.b	u.s1.b	u.s1.b
	u.s1.c	padding	padding	padding
	padding	padding	padding	padding

e) What is the size of the two dimensional array `arr`? If `arr[0][0]` is at address 3000, what is the address of `arr[3][5]`? And what is the address of `arr[3][5].s1.b`?

Solution: The size of the array is 24*5*10 = 1200 bytes. The starting address of `arr[3][5]` is 3000 + 24*(3*10 + 5) = 3840. The address of `arr[3][5].s1.b` is 3840 + 8 = 3848.

Each box below is 24 bytes.

3000	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	arr[0][4]	arr[0][5]	arr[0][6]	arr[0][7]	arr[0][8]	arr[0][9]
3240	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	arr[1][4]	arr[1][5]	arr[1][6]	arr[1][7]	arr[1][8]	arr[1][9]
3480	arr[2][0]	arr[2][1]	arr[2][2]	arr[2][3]	arr[2][4]	arr[2][5]	arr[2][6]	arr[2][7]	arr[2][8]	arr[2][9]
3720	arr[3][0]	arr[3][1]	arr[3][2]	arr[3][3]	arr[3][4]	arr[3][5]	arr[3][6]	arr[3][7]	arr[3][8]	arr[3][9]
3960	arr[4][0]	arr[4][1]	arr[4][2]	arr[4][3]	arr[4][4]	arr[4][5]	arr[4][6]	arr[4][7]	arr[4][8]	arr[4][9]

Problem 8 Given the following C++ code, answer the questions below.

```
1  class A {
2    public:
3      int val() { return 10;}
4      void p() { cout << "A" << val() << endl;}
5  };
6  class B : public A {
7    public:
8      int val() { return 8;}
9  };
10 class C: public B {
11   public:
12     void p() { cout << "C" << val() << endl;}
13 };
14 void f(A *ap) {
15   ap->p();
16 }
17 int main() {
18   f(new A());
19   f(new B());
20   f(new C());
21   return 0;
22 }
```

a) What is the output of this program? Specify for method `val()` and `p()` respectively whether they use dynamic method binding or static method binding.
Solution: `val()`: static method binding, `p()`: static method binding

```
A10
A10
A10
```

b) If we add the attribute **virtual** only to line 3 for `val()` method, what is the output of this modified program? Specify for method `val()` and `p()` respectively whether they use dynamic method binding or static method binding.
Solution: `val()`: dynamic method binding, `p()`: static method binding

```
A10
A8
A8
```

c) If we add the attribute **virtual** only to line 4 for `p()` method, what is the output of this modified program? Specify for method `val()` and `p()` respectively whether they use dynamic method binding or static method binding.
Solution: `val()`: static method binding, `p()`: dynamic method binding

```
A10
A10
C8
```

d) If we add the attribute **virtual** to both line 3 and line 4 for `val()` and `p()` methods, what is the output of this modified program? Specify for method `val()` and `p()` respectively whether they use dynamic

method binding or static method binding.

Solution: `val()`: dynamic method binding, `p()`: dynamic method binding

---

```
A10
A8
C8
```

Problem 9 Consider the following code that calculates sum of squares from 0 to n (assuming $n \geq 0$).

```
int SumOfSquares(int n){
    if (n == 0)
        return 0;
    else
        return n * n + SumOfSquares(n - 1);
}
```

1. Explain why this implementation of `SumOfSquares` will take a long time to run and uses a lot of memory (sometimes even run out of memory) with a large n.

   Solution: The current implementation requires us to keep activation record/stack frames for every recursive call of sumsquare because we will need to combine the result of the recursive call with current instance's n value to get our result. These are not tail-recursive calls. So each round of recursive call, we need to (1) allocate stack frames which takes up lots of memory; (2) complete the calling sequences which maintains the call stack including saving registers, saving return values on stack and so on, which takes a lot of time.

2. Rewrite this function `SumOfSquares` into an equivalent tail-recursive function.

   Solution:

```
int SumOfSquares(int n) {
  return SumOfSquaresHelper(0, n);
}
int SumOfSquaresHelper(int a, int n) {
  if (n<=0)
    return a;
  else
    return SumOfSquaresHelper(a+n*n, n-1);
}
```

Problem 10

a) Indicate whether each of the following will unify or not. If unification will succeed, indicate how each variable will be bound. If unification will fail, briefly (a single phrase or sentence) explain why.

- `f(g(X, Y), h(Y)) = f(g(r(T), 2), Z).`
  Solution: `X = r(T), Y = 2, Z = h(2)`
- `[X, [a, b], X|Z] = [a, [X|Z], a, b].`
  Solution: `X = a, Z = [b]`
- `[X, [a, b], X|Z] = [a, [X, Z], a, b].`
  Solution: False. `Z` cannot be `b` and `[b]` at the same time.
- `[X, [a, b], X|_] = [a, [X, _], a, b].`
  Solution: `X = a`

b) Write a predicate `element_at(E,Ys,N)` that succeeds if `E` is the N^{th} element of list `Ys`. You can assume that `Ys` is 0-indexed, and `N` is less than the length of `Ys`. For example:

```
?- element_at(3,[1,2,3],2).   % true
?- element_at(1,[1,2,3],0).   % ture
?- element_at(1,[1,2,3],1).   % false
```

Solution:

```
element_at(E, [E|_], 0).
element_at(E, [_|Ys], N) :- N > 0, M is N - 1, element_at(E, Ys, M).
```

c) Write a predicate `insert_at(E,Ys,N,Zs)` that succeeds if `Zs` is the list `Ys` with `E` inserted at index `N`, i.e., insert `E` at index `N` in `Ys`. You can assume that `Ys` is 0-indexed, and `N` is less than the length of `Ys`. For example:

```
?- insert_at(3,[1,2,3],2,[1,2,3,3]).  % true
?- insert_at(1,[1,2,3],0,[1,1,2,3]).  % true
?- insert_at(a,[1,2,3],1,[1,a,2,3]).  % true
?- insert_at(1,[1,2,3],0,[1,2,3]).    % false
```

Solution:

```
insert_at(E, Ys, 0, [E|Ys]).
insert_at(E, [Y|Ys], N, [Y|Zs]) :- N > 0, M is N - 1, insert_at(E, Ys, M, Zs).
```